

A Bioinspired Metaheuristic Algorithm: Ant Colony Optimization

Presented By: Aaliyah Hänni

Advised By: Valentin Wyss

Course: 61641 Seminar Applied Optimization

Date: 21.05.2026

Index

1

Define the Problem Space

2

Ant Behaviors

3

Ant Colony Optimization (ACO)

4

Optimizing Swiss Hiking Routes with ACO

5

Ant Colony Optimization for Continuous Domains (ACO_R)

6

Appendix: Ant Colony Optimization for Mixed Variable Domains (ACO_{MV})

7

Appendix: General Improvements to Ant Colony Optimization

Optimization

Given a function $f:S \rightarrow \mathbb{R}$, find $x^* \in S$, such that

$$\forall x \in S, f(x^*) \geq f(x) \text{ (maximization)}$$

$$\forall x \in S, f(x^*) \leq f(x) \text{ (minimization)}$$

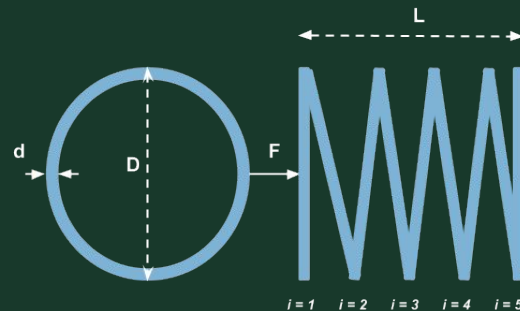
Discrete Optimization



Continuous Optimization



Mixed-variable optimization



Combinatorial Optimization Problems

Subset of discrete optimization problems, where solutions are combinations of
a finite set of elements

example: Find shortest path that visits every node exactly once and returns to the start.

- The number of unique paths is

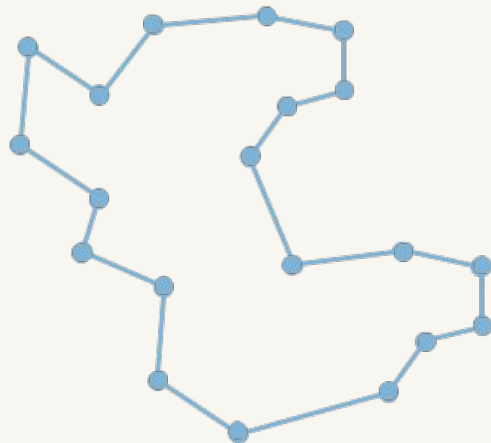
$$\frac{(n-1)!}{2}$$

- Brute force time complexity is

$$O(n!)$$

- With 20 nodes, total possible paths

$$\frac{(20-1)!}{2} = \frac{19!}{2} \approx 6.08 \times 10^{16}$$



Traveling Salesman Problem (TSP)

- Goal: find shortest route visiting each city once
- Cities are nodes and travel choices are edges
- Objective Function:

$$\min_r(\text{distance of route } r)$$

- NP-Hard
 - Hard to solve
 - Hard to verify



Metaheuristic Algorithms: Stochastic Approaches

Methods for solving optimization problems:

1. **exact methods:** guarantee optimality but may be slow
2. **heuristic methods:** aim for good solutions in reasonable time

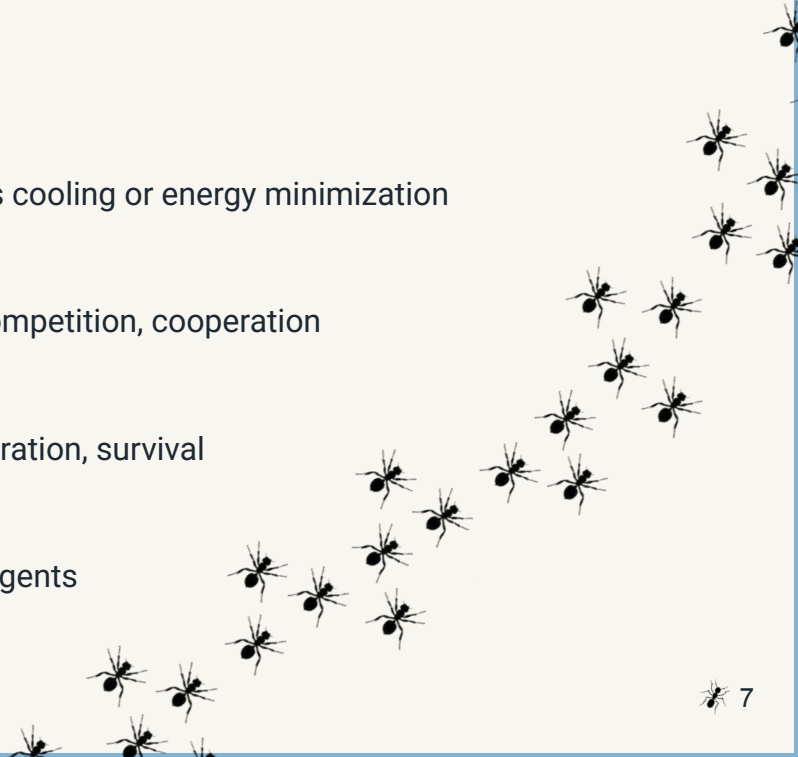
εὕρισκω (heurískō): to
happen upon by chance

Metaheuristics: general strategies that guide heuristic search

Bio-Inspired Algorithms

Computation techniques inspired by natural processes

- **Evolutionary algorithms:** selection, crossover, mutation
- **Physics-inspired algorithms:** physical processes such as cooling or energy minimization
- **Human/social-inspired algorithms:** learning, teaching, competition, cooperation
- **Ecology-inspired algorithms:** predator-prey behavior, migration, survival
- **Swarm algorithms:** collective behavior of decentralized agents



Ant Behaviors

Stigmergy

Pheromone trails

Evaporation



Photo Source: Wikipedia Commons [Jidapa Promdech](#)

Binary Bridge Experiments

In 1990 Deneubourg created the **binary bridge**

experiment:

- Ants given two bridges to a food source
- Initially random fluctuations
- Over time, observed convergence to the shorter bridge

Behavior modeled by Goss:

- Monte Carlo simulations
- Demonstrating positive feedback through pheromone trails

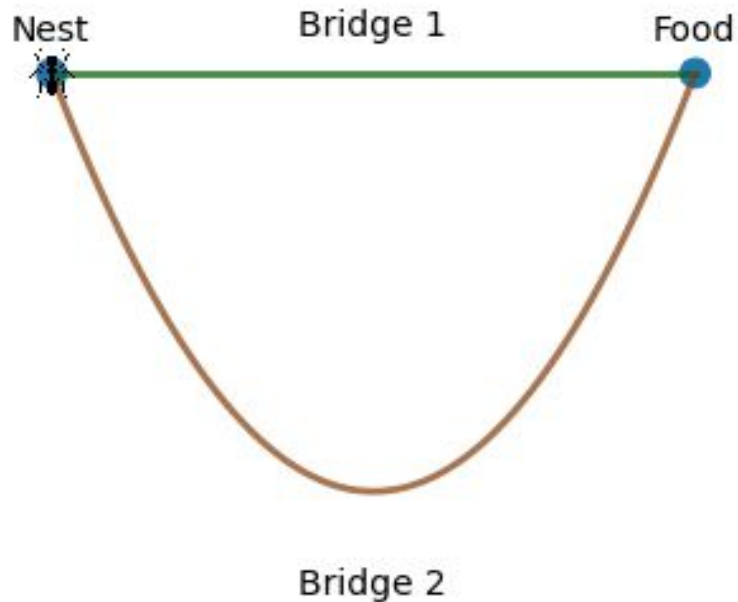
$$p_{1(m+1)} = \frac{(m_1 + k)^h}{(m_1 + k)^h + (m_2 + k)^h}$$

m_1 : pheromone strength on bridge 1

m_2 : pheromone strength on bridge 2

k : controls how random early choices are

h : controls the strength of positive feedback



Total choices so far

Bridge 1: 0

Bridge 2: 0

Pheromone

Bridge 1: 0.00

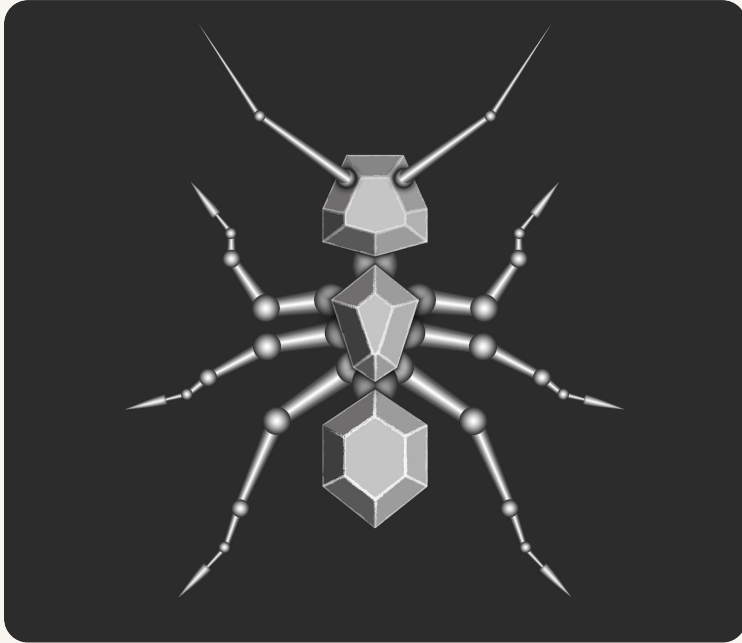
Bridge 2: 0.00

Probability

P(Bridge 1): 0.00

P(Bridge 2): 0.00

Ant Colony Optimization (ACO)



[Meta]heuristic that allows for polynomial complexity
First proposed by **Dorigo** in 1991 as a solution for solving combinatorial optimization problems

From real ants to artificial ants:

- Path choice becomes **probabilistic solution construction**
- **Pheromone** trails become artificial memory
- Pheromone **evaporation** prevents premature convergence
- Good paths are reinforced over time

Algorithm: Ant Colony Optimization (ACO)

procedure ACO is

Initialize step: set parameters, initialize
pheromone trail

while stopping condition not met **do**

 constructAntSolutions()

 store best solution found so far

 daemonActions() (optional)

 pheromoneUpdate()

repeat

return best solution

end procedure

Complexity:

$$O(NC \ m \ n^2)$$

TSP ($n = m$):

$$O(NC \ n^3)$$

NC : cycles counter

n : number of solution variables

m : number of ants



Step 1: constructAntSolutions()

$$p(c_{ij}|s^p) = \frac{\tau_{ij}^\alpha \cdot \eta(c_{ij})^\beta}{\sum_{c_{il} \in \mathbf{N}(s^p)} \tau_{il}^\alpha \cdot \eta(c_{il})^\beta} \quad \forall c_{ij} \in \mathbf{N}(s^p)$$

- s^p : current partial solution
- τ_{ij} : pheromone value
- $\eta(\cdot)$: a function that assigns a heuristic value to each solution component $c_{ij} \in \mathbf{N}(s^p)$
- $\mathbf{N}(s^p)$: is the set of feasible neighbors
- $\alpha : \alpha \in \mathbb{R} \geq 0$
- $\beta : \beta \in \mathbb{R} \geq 0$



Step 2: daemonActions()

Daemon actions are **problem-specific** improvements

Improved solutions can guide **pheromone updates**

example: 2-opt Local Search



Daemon: Plato defined them as somewhere between gods and men

Source: R. von Lichtenberg - R. von Lichtenberg Prehistoric Greece. - Saint-Petersburg.: PP Soikin, 1914. - ISBN 180 (p. 120)

Step 3: pheromoneUpdate()

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \rho \sum_{s \in S_{upd} | c_{ij} \in s} F(s)$$

τ_{ij} : amount of pheromone between i and j

ρ : $\rho \in (0, 1]$, parameter evaporation rate

s : solution set

S_{upd} : the set of solutions that are used for the update

c_{ij} : component from i to j

$F(\cdot)$: fitness function

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \sum_{k=1}^m \Delta\tau_{ij}^k$$

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k} & \text{if ant } k \text{ uses edge } (i, j) \\ 0 & \text{otherwise} \end{cases}$$

m : number of ants

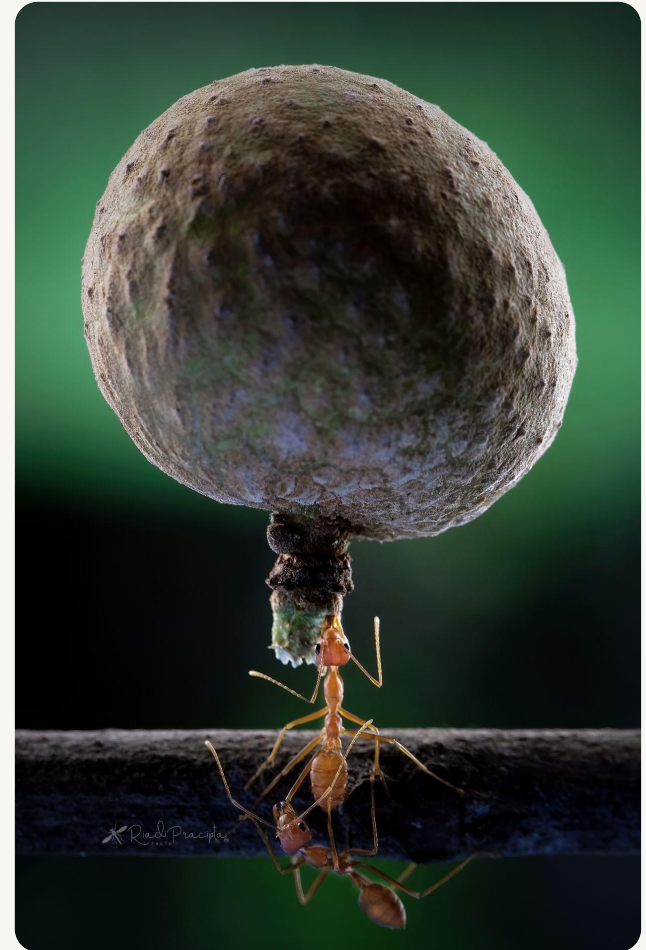
$\Delta\tau_{ij}^k$: amount of pheromone deposited by the k th ant

Q : constant pheromone strength

L_k : cost associated with k th ants' path

Motivation

- Good results
- Adaptive
- Iterative improvements
- Non exhaustive



Optimizing Swiss Hiking Routes with Ant Colony Optimization

Route Visualization



ACO for Continuous Domains (ACO _{$\mathbb{R}$})

- Introduced by Krzysztof Socha (2009)
- Extends ACO to continuous search spaces

Core Differences

- Replaces discrete component choices with continuous sampling

ACO: choose $c_{ij} \in N(s_p)$

ACO _{$\mathbb{R}$} : sample $x \sim P(x)$

- Uses a ranked solution archive
- Builds PDFs using Gaussian kernels around good solutions

ACO_R Algorithm

procedure ACO_R **is**

initialize solution archive with k random solutions

while stopping condition not met **do**

 evaluate solutions using objective function

 sort archive

 assign weights and probabilities to each

for each m ant

 generate new solution from Gaussian kernel

 daemonActions() (optional)

 add m solutions to archive

 sort $k + m$ archive

 remove worst m solutions

repeat

return best solution

end procedure



Complexity

$$O\left(NC\left[k^2n + mn + mC_f + (k+m)\log(k+m)\right]\right)$$

NC : number of iterations

k : solution archive size

m : number of ants

n : number of decision variables

C_f : cost of evaluating the objective function $f(x)$

1. Solution Construction:

Step 1: a single ant chooses probabilistically a solution from the solution archive.

$$p_j = \frac{w_j}{\sum_{r=1}^k w_r}$$

$$w_j = \frac{1}{qk\sqrt{2\pi}} e^{\frac{-(j-1)^2}{2q^2k^2}}$$

p_j : probability of an ant choosing a solution j

w_j : weight associated with solution j

q : parameter of the algorithm

k : archive size

2. Generate New Solutions

Step 2: Iterating through the chosen solution with $i = 1, \dots, n$ problem variables. the ant takes the i th variable of the chosen j th neighborhood and samples its neighborhood using a PDF. Created once per ant, per iteration.

$$P(x) = g(x, \mu, \sigma) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$$\sigma_j^i = \xi \sum_{r=1}^k \frac{|s_r^i - s_j^i|}{k-1}$$

$$\mu_j^i = s_j^i$$

Note: The PDF can be any, in the text a Gaussian function was used, defined with the above mean and standard deviation.

- s_j^i : solution variable, $i=1, \dots, n$ of the chosen j th solution
- σ_j^i : average distance between the i th variable of the solution s_j^i , and the i th variable of the other solutions in the archive. Calculate the average distance from chosen s_j^i to other solutions in the archive.
- ξ : parameter that has an effect similar to pheromone evaporation in ACO, higher ξ gives a lower convergence speed. evaporation multiplier.

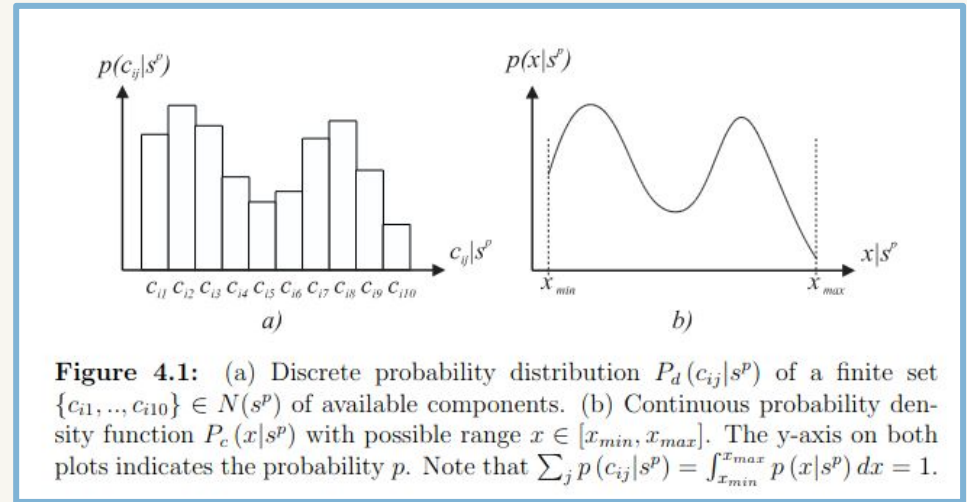
Probability Density Function (PDF)

Probability Density Function (PDF): represents the relative likelihood of a continuous random variable taking a specific value. Properties:

- Non-negativity: $f(x) \geq 0$ for all x
- Total Area Equals 1: $\int_{-\infty}^{\infty} f(x) dx = 1$
- Interval Probability: $P(a < X \leq b) = \int_a^b f(x) dx$
- Individual Probability: $P(X=a) = 0$

1. Candidate solutions are constructed in a probabilistic way using a probability distribution over the search space;

2. The candidate solutions are used to modify the probability distribution in a way that is deemed to bias future sampling toward high quality solutions



Pheromone Model

		1	2	...	i	...	n		
s_1	1	s_1^1	s_1^2	...	s_1^i	...	s_1^n	$f(s_1)$	ω_1
s_2	2	s_2^1	s_2^2	...	s_2^i	...	s_2^n	$f(s_2)$	ω_2
		.	.	.	.	.	.	.	.
		.	.	.	.	.	.	.	.
		.	.	.	.	.	.	.	.
		.	.	.	.	.	.	.	.
s_j	j	s_j^1	s_j^2	...	s_j^i	...	s_j^n	$f(s_j)$	ω_j
		.	.	.	.	.	.	.	.
		.	.	.	.	.	.	.	.
		.	.	.	.	.	.	.	.
		.	.	.	.	.	.	.	.
s_k	k	s_k^1	s_k^2	...	s_k^i	...	s_k^n	$f(s_k)$	ω_k

Figure 4.2: The structure of the solution archive. The solutions in the archive are sorted according to their quality (i.e., the value of the objective function $f(s)$), hence the position of a solution in the archive always corresponds to its rank.

Pheromone Model: used to bias the solution construction over time towards the search space containing high quality solutions

- ACO Pheromone: pheromone values are a finite set, discrete values related to the decision ants make, stored in a pheromone table
 - Pheromone model is an implicit memory of search history
- ACOR Pheromone: uses a solution archive to describe the pheromone distribution over the search space, this archive contains a number of complete solutions
 - Solution archive is an explicit memory of search history

Trivial Example with $\text{ACO}_{\mathbb{R}}$

Example: Minimize $f(x) = (x-5)^2$, for $x \in [0, 10]$

1. Generate a **solution archive** with random solutions
2. **For each solution**: calculate objective, sort, weights, and probabilities
3. **Generate new solutions** using a probability distribution centered closer to the best solutions
4. **Add new sample points** to the archive, calculate objectives for new, sort, discard the worst
5. Iterate 2-4 until stopping criteria

Example: Minimize $f(x) = (x-5)^2$, for $x \in [0, 10]$

1. Generate a **solution archive** with random solutions

s
4
8.2
3.14
1
9.9

Example: Minimize $f(x) = (x-5)^2$, for $x \in [0, 10]$

2. For each solution: calculate objective function then sort, calculate weights and probabilities, calculate μ and σ

s	f(s)	w_j	p_j	μ	σ
4	1	0.3989	0.570	4	1.745
8.24	30.4506	0.2420	0.346	3.14	1.8525
8.24	30.4506	0.0540	0.077	8.2	2.3775
1	16	0.0044	0.006	1	2.5475
9.9	24.01	0.0001	0.0002	9.9	3.015

Example: Minimize $f(x) = (x-5)^2$, for $x \in [0, 10]$

3. Generate new solutions using a probability distribution centered closer to the best solutions

s	p_j	$N(\mu, \sigma^2)$	$r \sim U(0,1)$
N(4, 1.745 ²)	0.570	N(4, 1.745 ²)	$0 \leq r < 0.570$
N(3.14, 1.8525 ²)	0.346	N(3.14, 1.8525 ²)	$0.570 \leq r < 0.916$
N(8.2, 2.3775 ²)	0.077	N(8.2, 2.3775 ²)	$0.916 \leq r < 0.993$
N(1, 2.5475 ²)	0.006	N(1, 2.5475 ²)	$0.993 \leq r < 0.999$
N(9.9, 3.015 ²)	0.002	N(9.9, 3.015 ²)	$0.999 \leq r \leq 1$

r	$N(\mu, \sigma^2)$	s_{new}
0.51	N(4, 1.745 ²)	6.1072
0.86	N(3.14, 1.8525 ²)	1.8719
0.803	N(3.14, 1.8525 ²)	1.6572
0.452	N(4, 1.745 ²)	5.6653
0.928	N(8.2, 2.3775 ²)	3.1347

Example: Minimize $f(x) = (x-5)^2$, for $x \in [0, 10]$

4. Add new sample points to the archive, calculate objectives for new, sort, discard the worst

s	f(s)
5.6653	0.4426
6.1072	3.2206
0.2	10.24
3.14	30.596
9.9347	24.09
6.1072	3.2206
6.8070	9.286
3.2114	30.596
1.8572	11.17
3.3347	3.496
5.6653	0.4426
9.9347	24.09

Example: Minimize $f(x) = (x-5)^2$, for $x \in [0, 10]$

5. Iterate 2-4 until stopping criteria

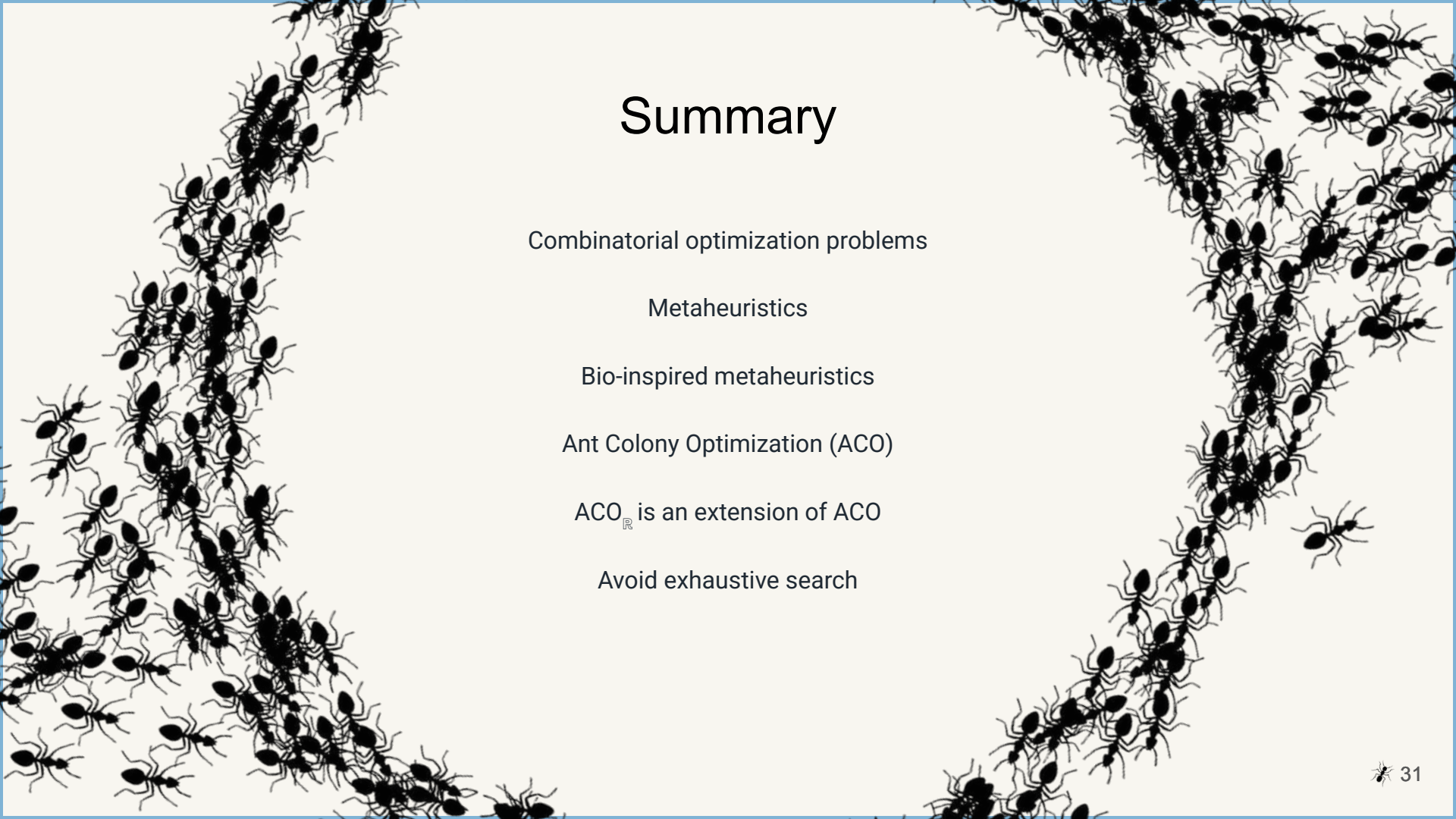
s
5.0023
5.6269
5.0000
6.5032
6.3350

Motivation



Benefits of ACO_R

- Does not require a differentiable objective function like in gradient descent and newton method
- Can handle nonlinear, multimodal, and black-box functions
- Can also be extended to mixed-variable problems

A large number of black ants are arranged in two dense, diagonal lines that run from the top-left corner to the bottom-right corner of the slide, framing the central text. The ants are depicted in various orientations, some facing left and some right, creating a sense of movement and swarm behavior.

Summary

Combinatorial optimization problems

Metaheuristics

Bio-inspired metaheuristics

Ant Colony Optimization (ACO)

ACO_R is an extension of ACO

Avoid exhaustive search

Appendix I: ACO for Continuous Domains Mixed Variable Optimization

Mixed-Variable Optimization

- Continuous Relaxation-based methods: relax the requirement for discrete variables, making it a continuous optimization problem (note cannot be done in the case of categorical discrete variables)
- Genetic Adaptive Search: metaheuristic genetic algorithm that allows for flexible approach to the representation of variables. The crossover is performed variable-by-variable, if discrete a binary crossover is performed, otherwise if continuous, the children are created using a two-modal probability distribution, whose peaks are aligned with the values of the given continuous variable of the parents. Similar handling is done with the mutation.

Appendix III: Improvements to ACO

- Improvements to the pheromone update mechanism (ACO)²
 - MIN-MAX: only the “best” ants update the pheromone, $t_{\min}$, $t_{\max}$ bounds
 - IB-update
 - BS-Update

Ant System

MAX-MIN Ant System

Ant Colony System: local pheromone update, diversifying the search performed reducing falling into local minima

Hyper-cube ACO

Population-based ACO

Appendix IV: Exact Methods

1. **exact methods:** methods that guarantee finding an optimal solution to a given problem (but not always in reasonable time)
 - a. Enumeration approaches: enumerate all possibilities. Sometimes able to eliminate possibilities by feasibility arguments
 - b. Lagrangian Relaxations and Decomposition Methods: take a set of complicating constraints into the objection function, removing the complicating constraints from the constraint set (can) results in an easier to solve sub-problem
 - c. Cutting Plane algorithms based on polyhedral combinatorics: a convex polyhedron can be defined as the intersection of finitely many half spaces, or as the convex hull plus the conical hull of some finite number of vectors or points. This implies the existence of a finite system of linear inequalities whose solution set coincides with the convex hull of the mixed-integer points in S .

Sources

1. Ant System: Optimization by a Colony of Cooperating Agents (1996)
2. <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=484436>
[Ant Colony Optimization](#) (2006) by Marco Dorigo, Mauro Birattari, and Thomas Stützle
3. [Ant colony optimization for continuous domains](#) (2008) by Krzysztof Socha, Marco Dorigo
4. Socha Thesis: <https://iridia.ulb.ac.be/~mdorigo/HomePageDorigo/thesis/phd/SochaPhDThesis.pdf>
5. Colony-Enhanced Recurrent Neural Architecture Search: Collaborative Ant-Based Optimization (2024)
<https://arxiv.org/html/2401.17480v1>
6. [A conceptual comparison of several metaheuristic algorithms on continuous optimisation problems](#) (2020) by Absalom E. Ezugwu et. al.
7. [Bio-inspired optimization: metaheuristic algorithms for optimization](#) (2020) by Pravin S. Game, Dr. Vinod Vaze, Dr. Emmanuel M.
8. <https://link.springer.com/article/10.1007/bf01417909>
9. <https://www.random.org/gaussian-distributions/>

Thanks OpenAI !

AI Usage:

- Finding a data source
- Styling my webapp (.css)
- Creating Monti Python GIF visualization
- Simple explanations of complex concepts

s
4
8.2
3.14
1
9.9

s	f(s)
4	1
8.2	10.24
3.14	3.4596
1	16
9.9	24.01

s	f(s)
4	1
3.14	3.4596
8.2	10.24
1	16
9.9	24.01

s	f(s)	w_i $N(1, qk) = N(1, 1)$
4	1	0.3989
3.14	3.4596	0.2420
8.2	10.24	0.0540
1	16	0.0044
9.9	24.01	0.0001

s	f(s)	w _j	p _j
4	1	0.3989	0.570
3.14	3.4596	0.2420	0.346
8.2	10.24	0.0540	0.077
1	16	0.0044	0.006
9.9	24.01	0.0001	0.0002

s	mean	sd
4	4	1.745
3.14	3.14	1.8525
8.2	8.2	2.3775
1	1	2.5475
9.9	9.9	3.015

s	p_j	$N(u, sd^2)$	Interval $r \sim U(0,1)$
4	0.570	$N(4, 1.745^2)$	$0 \leq r < 0.570$
3.14	0.346	$N(3.14, 1.8525^2)$	$0.570 \leq r < 0.916$
8.2	0.077	$N(8.2, 2.3775^2)$	$0.916 \leq r < 0.993$
1	0.006	$N(1, 2.5475^2)$	$0.993 \leq r < 0.999$
9.9	0.0002	$N(9.9, 3.015^2)$	$0.999 \leq r \leq 1$

r	$N(u, sd^2)$	s_new
0.51	$N(4, 1.745^2)$	6.1072
0.86	$N(3.14, 1.8525^2)$	1.8719
0.803	$N(3.14, 1.8525^2)$	1.6572
0.452	$N(4, 1.745^2)$	5.6653
0.928	$N(3.14, 1.8525^2)$	3.1347

s	f(s)
4	1
8.2	10.24
3.14	3.4596
1	16
9.9	24.01
6.1072	1.226
1.8719	9.785
1.6572	11.17
5.6653	0.4426
3.1347	3.479

s	f(s)
5.6653	0.4426
4	1
6.1072	1.226
3.14	3.4596
3.1347	3.479
1.8719	9.785
8.2	10.24
1.6572	11.17
1	16
9.9	24.01

s	f(s)
5.6653	0.4426
4	1
6.1072	1.226
3.14	3.4596
3.1347	3.479